Both changes are due to the fact that computing the cost function and the transitions (by cloning a graph embedding) were particularly computationally-heavy. Of course, this is partly due to using a general-purpose version of simulated annealing, it could be optimized by writing an ad-hoc version that doesn't need to clone the whole embedding, but just remember what was changed, and what was the cost of current solution (and maybe also computing the delta based on the changes, not the whole solutions: for instance, just compute the number of intersections for the vertex moved before and after the change).

Anyway, I would like to avoid focusing on these optimizations here: don't get me wrong, optimization is crucial, especially for production-ready code, but early optimization can also get in the way of improving your algorithm or of your learning process – premature optimization is, still, the source of all evil (well, maybe not all of it, but a good chunk!).

In this case, I preferred providing simple, clean non-optimized code rather than more obscure (though better performing) routines.

The next step will be adding a larger-range search: swapping two vertices' positions in 10% of the iterations, so that 90% of the times we apply the short-range transition. How do you think it went? Well, just poorly: the average number of intersections grew to `22.89`; this, however, wasn't unexpected: if you think about it, a complete graph is *completely symmetrical*, so that swapping two vertices is totally useless! Even worse, it was detrimental because we were wasting 10% of the iterations, hence the poorer result.